

HDOC:-30

Name:-Arayana Khare

Batch:-15

Algorithm

1. Start.
2. Read the size **n** of the array. Ensure $n > 0$.
3. Read **n** array elements.
4. Display the menu:
 1. Bubble Sort and Binary Search
 2. Insertion and Deletion
 3. Right Rotation by **k** positions
 4. Exit
5. Read the user's choice.
6. Use a **switch** statement:
 1. **Choice 1:** Sort the array in ascending order using Bubble Sort. Read the search element and locate it using Binary Search.
 2. **Choice 2:** Ask whether the user wants insertion or deletion. For insertion, read a valid position and new element and shift elements to the right. For deletion, read a valid position and shift remaining elements to the left.
 3. **Choice 3:** Read **k** and rotate the array to the right **k** times.
 4. **Choice 4:** Exit.
 5. Otherwise, display an error message.
7. Repeat until the user selects Exit.
8. Stop.

Test-Case Table

C Program

```
#include <stdio.h>
int main()
{
int a[100];
```

Test	Operation	Input	Expected Output
1	Bubble Sort + Search	5: 40 10 30 20 50, key 30	10 20 30 40 50; 30 found
2	Binary Search	5: 40 10 30 20 50, key 60	Sorted array; 60 not found
3	Insertion	10 20 30 40, insert 25 at position 3	10 20 25 30 40
4	Deletion	10 20 30 40 50, delete position	10 20 40 50

5	Right Rotation	3 1 2 3 4 5, k = 2	4 5 1 2 3
6	Boundary Rotation	1 2 3 4 5, k = 5	1 2 3 4 5
7	Invalid menu	Choice 7	Invalid choice
8	Exit	Choice 4	Program terminated

```

int n, choice, i, j;
/* Input array size */
do
{
printf("Enter the number of elements: ");
scanf("%d", &n);
if (n <= 0 || n > 100)
printf("Enter a size between 1 and 100.\n");
} while (n <= 0 || n > 100);
printf("Enter %d elements:\n", n);
for (i = 0; i < n; i++)
scanf("%d", &a[i]);
do
{
printf("\n===== HDOC DAY-30 =====\n");
printf("1. Bubble Sort and Binary Search\n");
printf("2. Insertion and Deletion\n");
printf("3. Right Rotation by k positions\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
{
/* Bubble Sort and Binary Search */
case 1:
{
int temp, key;
int low, high, mid;
int found = -1;
/* Bubble Sort */
for (i = 0; i < n - 1; i++)
{
for (j = 0; j < n - 1 - i; j++)
{
if (a[j] > a[j + 1])
{
temp = a[j];

```

```
a[j] = a[j + 1];
a[j + 1] = temp;
}
}
}
printf("Sorted array:\n");
for (i = 0; i < n; i++)
printf("%d ", a[i]);
printf("\n");
/* Binary Search */
printf("Enter element to search: ");
scanf("%d", &key);
low = 0;
high = n - 1;
while (low <= high)
{
mid = (low + high) / 2;
if (a[mid] == key)
{
found = mid;
break;
}
else if (key < a[mid])
{
high = mid - 1;
}
else
{
low = mid + 1;
}
}
if (found != -1)
printf("%d found at position %d.\n",
key, found + 1);
else
printf("%d not found in the array.\n",
key);
```

```

break;
}
/* Insertion and Deletion */
case 2:
{
int operation, position, element;
printf("\n1. Insertion\n");
printf("2. Deletion\n");
printf("Enter your choice: ");
scanf("%d", &operation);
if (operation == 1)
{
if (n == 100)
{
printf("Array is full. Insertion not
possible.\n");
break;
}
printf("Enter element to insert: ");
scanf("%d", &element);
do
{
printf("Enter position (1 to %d): ",
n + 1);
scanf("%d", &position);
if (position < 1 || position > n + 1)
printf("Invalid position. Enter
again.\n");
} while (position < 1 || position > n +
1);
for (i = n; i >= position; i--)
{
a[i] = a[i - 1];
}
a[position - 1] = element;

```

```

n++;
printf("Array after insertion:\n");
for (i = 0; i < n; i++)
printf("%d ", a[i]);
printf("\n");
}
else if (operation == 2)
{
do
{
printf("Enter position to delete (1
to %d): ", n);
scanf("%d", &position);
if (position < 1 || position > n)
printf("Invalid position. Enter
again.\n");
} while (position < 1 || position > n);
element = a[position - 1];
for (i = position - 1; i < n - 1; i++)
{
a[i] = a[i + 1];
}
n--;
printf("Deleted element = %d\n",
element);
printf("Array after deletion:\n");
for (i = 0; i < n; i++)
printf("%d ", a[i]);
printf("\n");
}
else
{
printf("Invalid operation.\n");
}

```

```

}
break;
}
/* Right Rotation */
case 3:
{
int k, r, last;
printf("Enter number of positions to rotate:
");
scanf("%d", &k);
if (k < 0)
{
printf("k must be non-negative.\n");
break;
}
/* Avoid unnecessary rotations */
k = k % n;
for (r = 0; r < k; r++)
{
last = a[n - 1];
for (i = n - 1; i > 0; i--)
{
a[i] = a[i - 1];
}
a[0] = last;
}
printf("Array after right rotation:\n");
for (i = 0; i < n; i++)
printf("%d ", a[i]);
printf("\n");
break;
}

```

```

case 4:
printf("Program terminated.\n");
break;
default:
printf("Invalid choice. Enter a choice from 1
to 4.\n");
}
} while (choice != 4);
return 0;
}

```

Execution 1 — Bubble Sort + Binary Search

```

Enter the number of elements: 4
Enter 4 elements:
10 20 30 40

===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 1
Sorted array:
10 20 30 40
Enter element to search: 30
30 found at position 3.

```

Execution 2 — Insertion

```

===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 2

1. Insertion
2. Deletion
Enter your choice: 1
Enter element to insert: 25
Enter position (1 to 5): 3
Array after insertion:
10 20 25 30 40

```

Execution 3 — Deletion

```
===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 2

1. Insertion
2. Deletion
Enter your choice: 2
Enter position to delete (1 to 5): 3
Deleted element = 25
Array after deletion:
10 20 30 40
```

Execution 4 — Right Rotation

```
===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 3
Enter number of positions to rotate: 2
Array after right rotation:
30 40 10 20

===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
```